Stop Prompting, Start Planning

Reliable Agentic AI with Embabel

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

Abstract

Stop Prompting, Start Planning - Reliable Agentic AI with Embabel

Large Language Models enable powerful new AI capabilities, but production-grade enterprise agents require more than prompting alone. Prompt chaining often results in non-deterministic behavior, limited traceability, and systems that are difficult to control and test.

This talk introduces Embabel, an open source agent framework for the JVM created by Rod Johnson, founder of the Spring Framework. Embabel enables controllable, explainable, and production-ready agentic systems directly within existing Java and Spring applications.

Rather than orchestrating behavior through prompts, Embabel applies Goal-Oriented Action Planning (GOAP). Agents are composed of strongly typed actions, goals, and domain models, allowing execution paths to be dynamically replanned when assumptions change or steps fail.

Stop Prompting, Start Planning

Reliable Agentic AI with Embabel

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

Who Am I?

Patrick Baumgartner

Technical Agile Coach @ **42talents**

Focus on the **development of software solutions with humans**

Coaching, Architecture, Development, Reviews, Training

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of **Voxxed Days Zurich** and **JUG Switzerland**, Java Champion, Oracle ACE Pro Java, ...

Agenda

Agenda

1. **The Problem** | Why prompt chains break in production
2. **The Framework** | Introducing Embabel & GOAP
3. **Demo 1** | KYC Verification Agent (`01-embabel-kyc-agent`)
4. **Enterprise Patterns** | Deep dive into production features
5. **Demo 2** | Ticket Triage with Replanning (`02-embabel-ticket-triage`)
6. **Demo 3** | Guardrails: PII Detection (`03-embabel-pii-guardrails`)
7. **Demo 4** | MCP Meeting Planner (`04-embabel-mcp-meeting-planner`)
8. **Beyond the Basics** | DICE, progressive tools, industry validation
9. **Q&A and Key Takeaways**

Why Should You Care?

95% of Enterprise AI Projects Fail

About **95% of enterprise generative AI pilots fail** to deliver measurable business value, largely due to integration, workflow, and organizational challenges rather than model performance.

Not because of bad models. **Because of bad engineering.**

The €2 Million Bug

A 4-step prompt chain for loan approval. Works in dev.

- **Step 3** hallucinates a credit score
- **Step 4** approves a €2M loan to a non-existent company
- **Your audit trail?** A *String*.

"You cannot build business systems
on one nine... for business,
one nine is 100% doesn't work."

— Rod Johnson

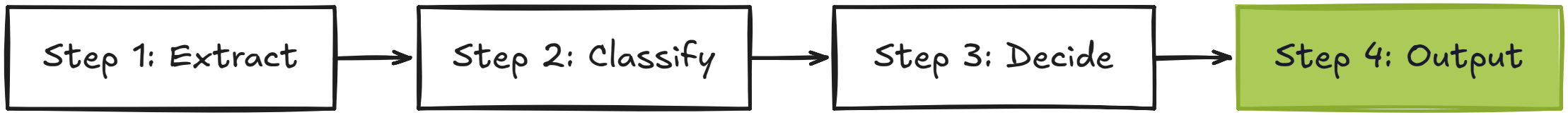
What If ...

*... your AI agent **planned like a chess engine** instead of **improvising like a jazz musician**?*

The Problem

Why Prompt Chains Break in Production

The Prompt Chain Pattern



Looks clean. Works in the demo. Ships to production.

Where It Falls Apart

- **Non-determinism**: Same input, different outputs
- **No traceability**: Which step caused the wrong answer?
- **Untestable**: Can't unit test a prompt chain
- **Fragile**: One model version change breaks everything
- **No recovery**: Step 3 fails → entire chain fails
- **No cost control**: Every step calls an LLM, no budget limits

The Central AI Team Anti-Pattern

- Separate AI teams build greenfield Python projects
- Business logic in Java microservices, AI in Jupyter notebooks
- Integration is an afterthought

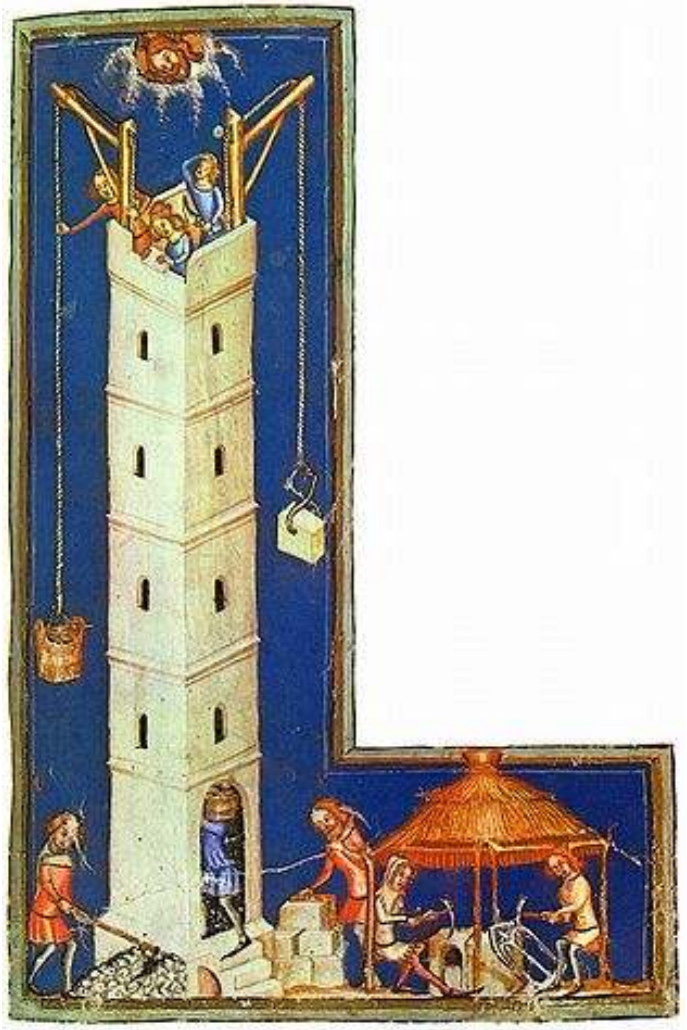
> *"The next phase of the AI revolution won't be written in Jupyter notebooks."*
> — Rod Johnson

What Enterprise Actually Needs

Requirement	Why
Controllability	Bounded, predictable behavior within policy limits
Explainability	"Why did the agent approve this claim?"
Testability	Standard JUnit, not vibe checks
Auditability	EU AI Act, DORA, FINMA require compliance-grade traces
Resilience	Replan when things go wrong
Cost Control	Budget limits per invocation, token tracking

The Framework

Introducing Embabel & GOAP



Der Turmbau zu Babel, from *Weltchronik* by Rudolf von Ems, ill. **Meister der Weltenchronik**, ca. 1370 · Bayerische Staatsbibliothek, Munich · Public Domain

Meet Embabel

- Open source agent framework for the **JVM**
- Created by **Rod Johnson**, founder of the Spring Framework
- Same philosophy: tame complexity with good abstractions
- Spring-native: works with your existing `@Controller`, `@Service`, `@Repository`, `@Configuration`

> *"Without agentic systems, we are more like alchemists than engineers, our prompts more like incantations."*
> — Rod Johnson

Getting Started

```
<dependency>
    <groupId>com.embabel.agent</groupId>
    <artifactId>embabel-agent-starter</artifactId>
    <version>${embabel-agent.version}</version>
</dependency>
```

```
@SpringBootApplication
@EnableEmbabelAgent
public class MyAgentApplication { ... }
```

- Works with your existing Spring Boot app
- No new runtime, no new infrastructure
- Start with one `@Agent` class

The Spring Parallel

2003	2026
EJBs: complex, heavyweight, untestable	Prompt chains: fragile, opaque, untestable
Spring: POJOs + IoC	**Embabel:** Typed actions + GOAP

From AI alchemy to AI systems engineering.

What Is GOAP?

- **Goal-Oriented Action Planning**, from game AI (F.E.A.R., 2005)
- You define: **Goals**, **Actions**, **Domain Model** (world state)
- The planner finds a path from current state to goal
- If a step fails → **replan dynamically**
- NOT an LLM deciding what to do: a **deterministic planner**

"Almost all enterprise Java developers have been killed by a GOAP AI at some point."

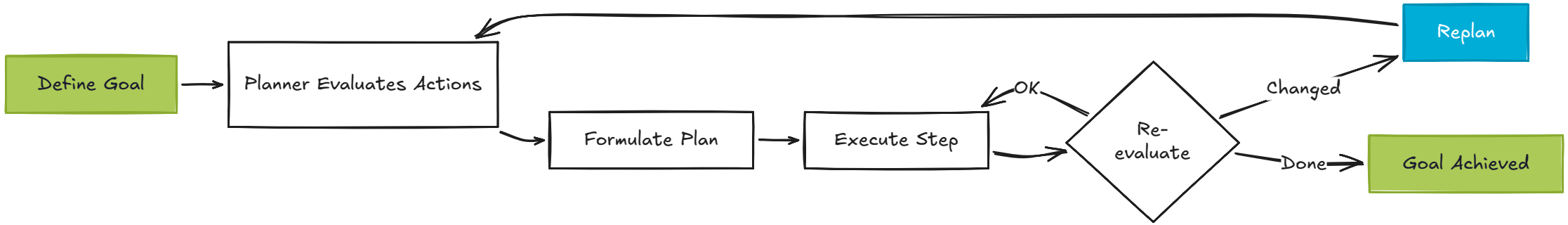
— Rod Johnson

Three Planner Types

Planner	Best For	Enterprise Example
GOAP	Deterministic goal-seeking	KYC verification, loan approval, claims
Utility AI	Exploration / event-driven	Customer chatbots, help desk
Supervisor	LLM selects actions as tools	Research tasks, open-ended analysis

All demos in this talk use **GOAP**.

How GOAP Works in Embabel



- Pre/post conditions inferred from **Java method signatures**
- Parameter types = preconditions, return type = postcondition
- Framework auto-injects knowledge cutoff dates + current date

Prompt Chaining vs. Embabel

Prompt Chaining	Embabel (GOAP)
LLM drives orchestration	Deterministic planner drives orchestration
String-based state passing	Strongly typed domain model
No recovery on failure	Dynamic replanning
Hard to test	Standard JUnit testing
No cost control	Budget limits + model mixing
New Python stack	Your existing JVM/Spring stack

Demo 1

KYC Verification Agent (`01-embabel-kyc-agent`)

Demo 1: KYC Verification Agent

2 Actions, Zero Orchestration Code

- **Domain:** Know Your Customer (banking)
- **3 records:** `KycRequest` , `KycScreening` , `KycAssessment`
- **2 actions:** `screenCustomer` → `assessRisk`
- **Key insight:** Embabel figured out the order from the types alone.

Demo 1: The Agent Class

```
@Agent(name = "KycVerificationAgent",
       description = "Screens a customer against risk indicators and produces a risk assessment.")
public class KycVerificationAgent {

    @Action(description = "Screen customer against PEP, sanctions, adverse media.")
    public KycScreening screenCustomer(KycRequest request, OperationContext context) {
        return context.ai()
                .withDefaultLlm()
                .creating(KycScreening.class)
                .withoutProperties("customerId")  // ← exclude internal ID
                .fromPrompt(prompt);
    }

    @AchievesGoal(description = "Produce a risk assessment.")
    @Action(description = "Assess overall KYC risk based on screening results.")
    public KycAssessment assessRisk(KycScreening screening, OperationContext context) {
        return context.ai()
                .withDefaultLlm()
                .creating(KycAssessment.class)
                .fromPrompt(prompt);
    }
}
```

Demo 1: REST Invocation

```
@PostMapping("/verify")
public KycAssessment verify(@Valid @RequestBody ApiKycRequest request) {
    return AgentInvocation
            .create(agentPlatform, KycAssessment.class)
            .invoke(request.toDomain());
}
```

One line. Type-safe. Testable. A real HTTP API.

Enterprise Patterns

Deep Dive into Production Features

Sealed Interfaces for Type-Safe Routing

```
sealed interface Signals permits SignalsOk, SignalsNeedsDeep {}
record SignalsOk(/* ... */) implements Signals {}
record SignalsNeedsDeep(/* ... */) implements Signals {}

@Action
public ConfidentCategory quickClassifyOk(SignalsOk signals) { /* ... */ }

@Action
public UncertainCategory quickClassifyNeedsDeep(SignalsNeedsDeep signals) { /* ... */ }
```

- The planner picks the right action based on the **runtime type**
- No `if/else` chains, no string matching
- The Java type system IS your router

"There are literally no magic strings anywhere in Embabel applications."

— Rod Johnson

Property Filtering + JSR-380 Validation

```
// Exclude properties from LLM context (correct fluent chain)
context.ai().withDefaultLlm()
    .creating(KycScreening.class)
    .withoutProperties("customerId")
    .fromPrompt(prompt);

// JSR-380 validation on LLM output — same as Spring MVC
record RiskAssessment(
    @NotBlank String category,
    @Min(0) @Max(100) int score
) {}
```

- Constraints auto-communicated to LLM in the prompt
- Validated on return, retry on failure
- **Zero learning curve**: same Bean Validation you already know

Domain Objects as LLM Tools

```
record Customer(Long id, String name, float balance) {
    @LlmTool(description = "Find balance of customer by id")
    float availableBalance() { return balance; }
}
```

- The entity IS the tool, DDD-aligned
- Context-specific, not a global function registry
- Retrieved entities carry their own tools

Model Mixing & No-LLM Actions

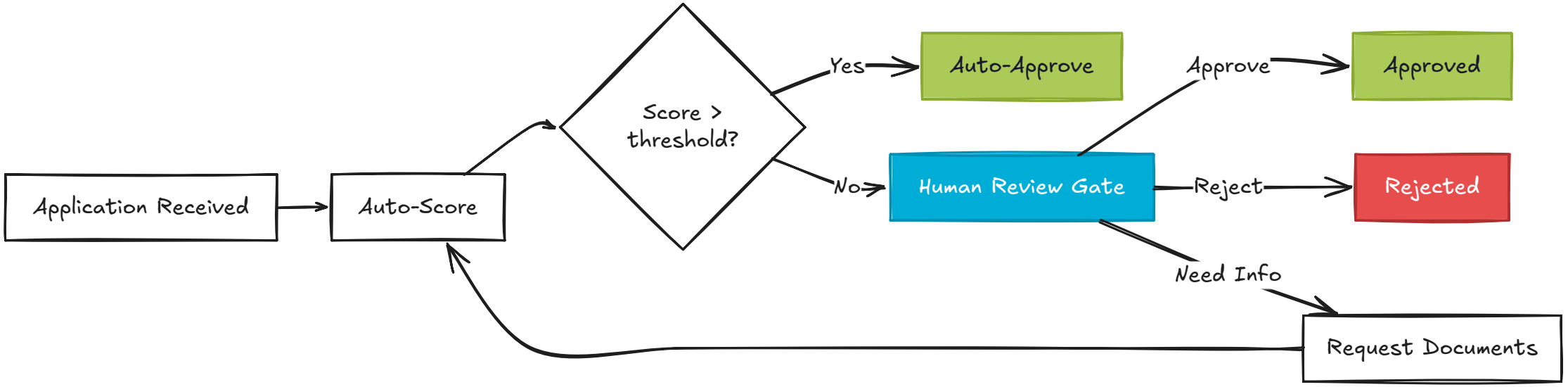
Step	Model	Why
Sensitive data	Local (Ollama)	PII never leaves infrastructure
Simple classification	Cheap cloud (gpt-4.1-mini)	Cost-effective
Complex reasoning	Frontier (gpt-5)	Maximum capability
Business rules	**No LLM**	Faster, cheaper,

```
context.ai().withLlm("qwen3").orElse("gpt4.1-mini")
  .creating(Result.class)
  .fromPrompt(prompt);
```

"The best kind of step is the step that doesn't use an LLM."

— Rod Johnson

Human-in-the-Loop with @State



- `@State` enables looping, branching, human intervention
- Critical for regulated workflows: loan approval, large transactions
- Regulators require human oversight for high-impact decisions

Testability at 4 Levels

Level	How
Unit	`FakeOperationContext` + `FakePromptRunner` → test deterministic logic
Integration	`embabel-agent-test` + `EmbabelMockitoIntegrationTest`

Observability: add `embabel-agent-starter-observability`

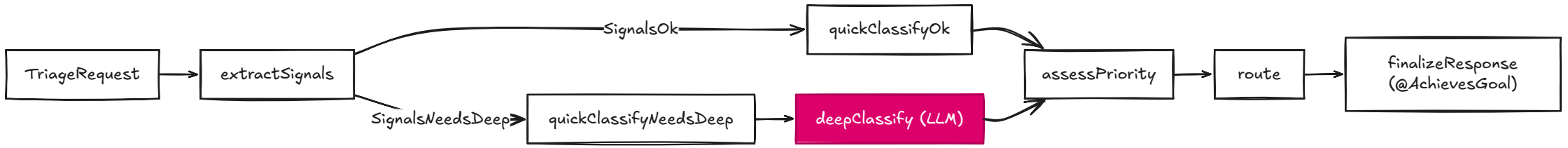
- Zero-code-change tracing: agent lifecycle, every action, LLM calls (tokens, latency)
- Integrates with Langfuse, OpenTelemetry, Jaeger, Grafana Tempo, …

Demo 2

Ticket Triage with Replanning (`02-embabel-ticket-triage`)

Demo 2: Ticket Triage Agent

Dynamic Replanning in Action



- **7 actions**, `1 @AchievesGoal`
- Sealed interfaces route execution automatically
- Deterministic fast-paths + LLM fallback

Demo 2: Flow A: Normal Incident

Request: `INC-101`: VPN outage

- category=INCIDENT, priority=HIGH, team=platform-ops
- **Fully deterministic. No LLM was called.**

Demo 2: Flow B: Replanning

Request: `REQ-202`: uncertain classification

- The planner detected uncertainty and **dynamically inserted** `deepClassify`
- **"I never coded this sequence. The planner found it."**

Demo 2: Flow C: Security Fast-Path

Request: SEC-777: data leak / phishing

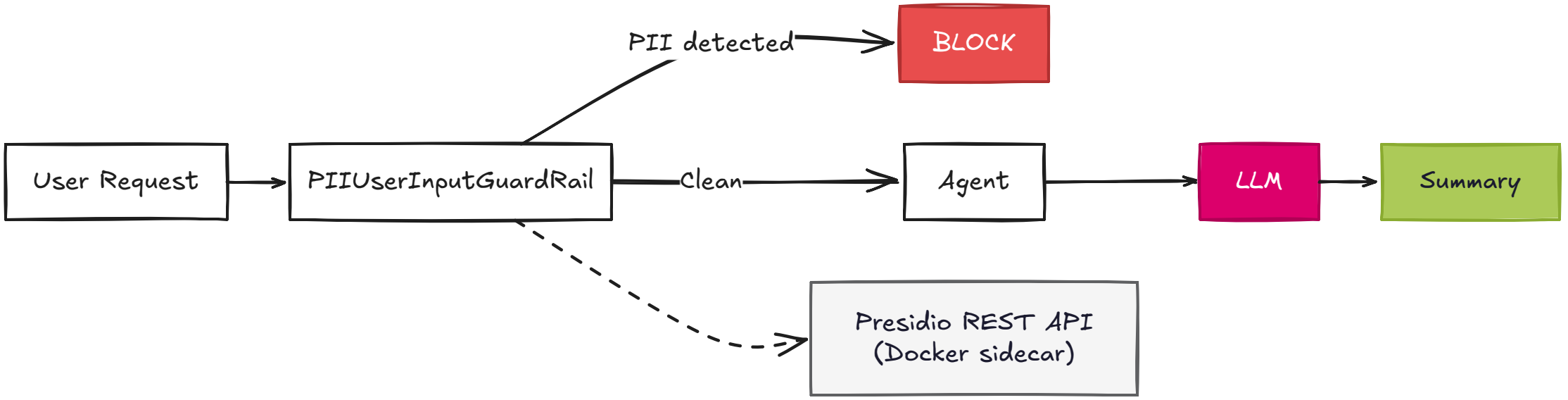
- category=SECURITY, priority=CRITICAL, team=security-incident
- **No LLM involved. Predictable, auditable.**

Demo 3

Guardrails: PII Detection (`03-embabel-pii-guardrails`)

Demo 3: PII Guardrails

The LLM Never Sees Your Data



- `UserInputGuardRail` : intercepts input **BEFORE** the LLM call
- Microsoft Presidio: specialized PII detection (not regex, not the LLM itself)
- Composable: `.withGuardRails(new PIIUserInputGuardRail(...))`
- Presidio sidecar is mapped as `localhost:5002` and readiness checked

Demo 3: The Guard Rail

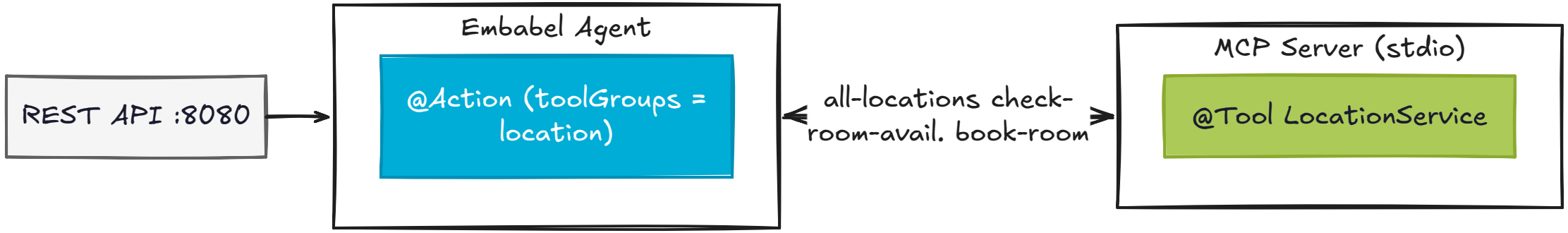
```
public class PIIUserInputGuardRail implements UserInputGuardRail {

    @Override
    public ValidationResult validate(String userInput, Blackboard blackboard) {
      List<AnalyzeResult> results = presidioClient.analyze(
        new AnalyzeRequest(userInput, "en", piiTypes));

      List<AnalyzeResult> highConfidence = results.stream()
        .filter(r -> r.score() >= 0.7).toList();

      if (highConfidence.isEmpty()) {
          return new ValidationResult(true, List.of());
      }

      return new ValidationResult(false, List.of(
        new ValidationError("PII_DETECTED",
          "PII detected: " + detected + ". Request blocked.",
          ValidationSeverity.CRITICAL)));
    }
}
```

Demo 4

MCP Meeting Planner (`04-embabel-mcp-meeting-planner`)

Demo 4: MCP Meeting Planner

Your Spring Services, Now AI-Ready



- MCP over stdio requires protocol-clean stdout from the tool process
- In this project, MCP server logs are routed to stderr to keep stdio JSON clean

Demo 4: MCP Tools → Agent Actions

```
// MCP Server: standard Spring @Service with @Tool
@Tool(name = "check-room-availability",
     description = "Check availability of a room at a location.")
public RoomAvailableResponse checkRoomAvailability(RoomAvailableRequest request) { }

// Embabel Agent: withToolGroups() connects the action to MCP tools
@Action(description = "Check availability of a room at the preferred location.")
public SuggestedRoom findRoomAtLocation(Location location, RoomRequest request,
                                        OperationContext context) {
    return context.ai()
        .withDefaultLlm()
        .withToolGroups("location")
        .creating(SuggestedRoom.class)
        .fromPrompt(prompt);
}
```

Demo 4: Consume vs Publish MCP

Two Different Paths

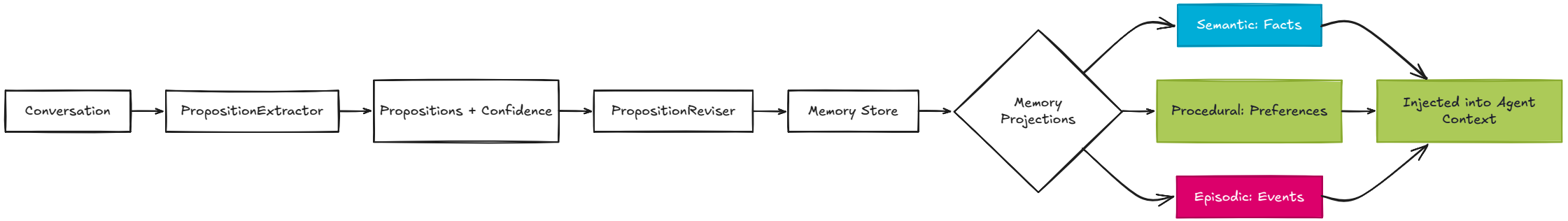
- **Consume MCP**: `context.ai().withToolGroups("location")` lets the agent call remote MCP tools in one LLM step
- **Publish an Embabel goal**: `@Export(remote = true)` turns a goal-achieving action into an MCP tool
- **Expose a plain Spring service**: `@Tool` publishes the service method directly

```
@AchievesGoal(export = @Export(remote = true))
public MeetingConfirmation bookMeetingRoom(...) { }
```

Beyond the Basics

DICE, Progressive Tools, and Industry Validation

DICE: Domain-Integrated Context Engineering



- Agent memory running on JVM, calling your own `CustomerRepository`
- Same transaction guarantees, same Spring context
- Preference extraction with confidence scores + decay rates

Agentic RAG: Search That Thinks

- `ToolishRag` : LLM **decides** when and how to search, not blind vector search
- Graph-powered RAG with Neo4j: structured knowledge (policy → clause → exception)
- No new stack. No Python bridge. Same Spring context.

"My goal was not to play catch-up or imitate Python."

— Rod Johnson

Progressive Tool Discovery

- Problem: LLMs choke on 100+ tools (49% accuracy on MCP evals, Anthropic)
- Solution: hierarchical tool grouping with `UnfoldingTool` / `@UnfoldingTools`
- LLM sees **4 facades** instead of 27 tools → O(log n) token consumption

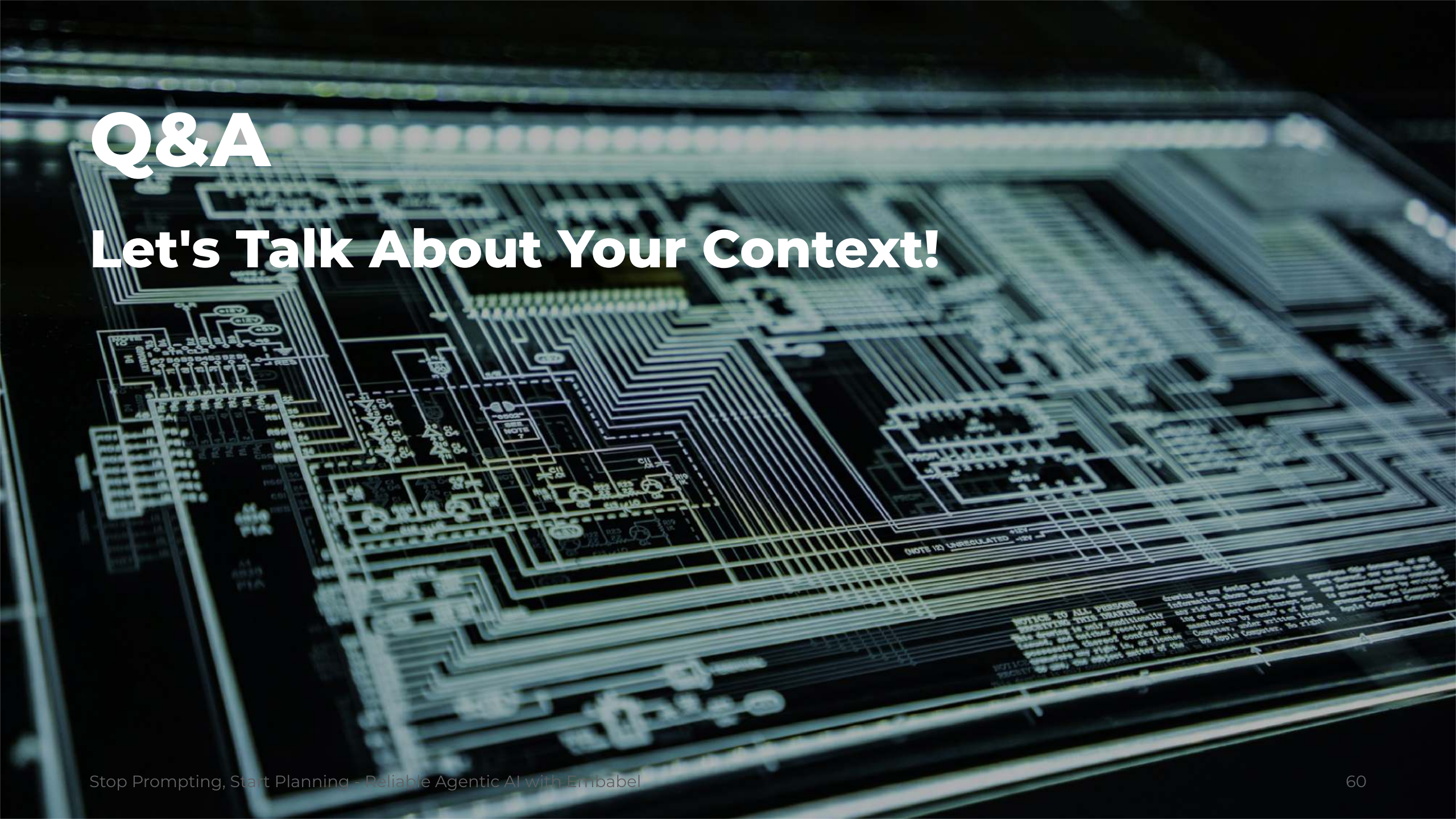
> **"Your bank has 200 microservices. You don't dump all their APIs into one prompt."**

Industry Validation

- **ThoughtWorks Technology Radar Vol. 34** (April 2026): "Structured output from LLMs" in **Adopt**
- **Gartner 2026:** Multiagent Systems and Domain-Specific Language Models as top strategic trends

"A very large proportion of enterprise needs are not chat related...
I think we overindexed on chat."

— Rod Johnson



Q&A

Let's Talk About Your Context!

Key Takeaways

5 Things to Remember

5 Things to Remember

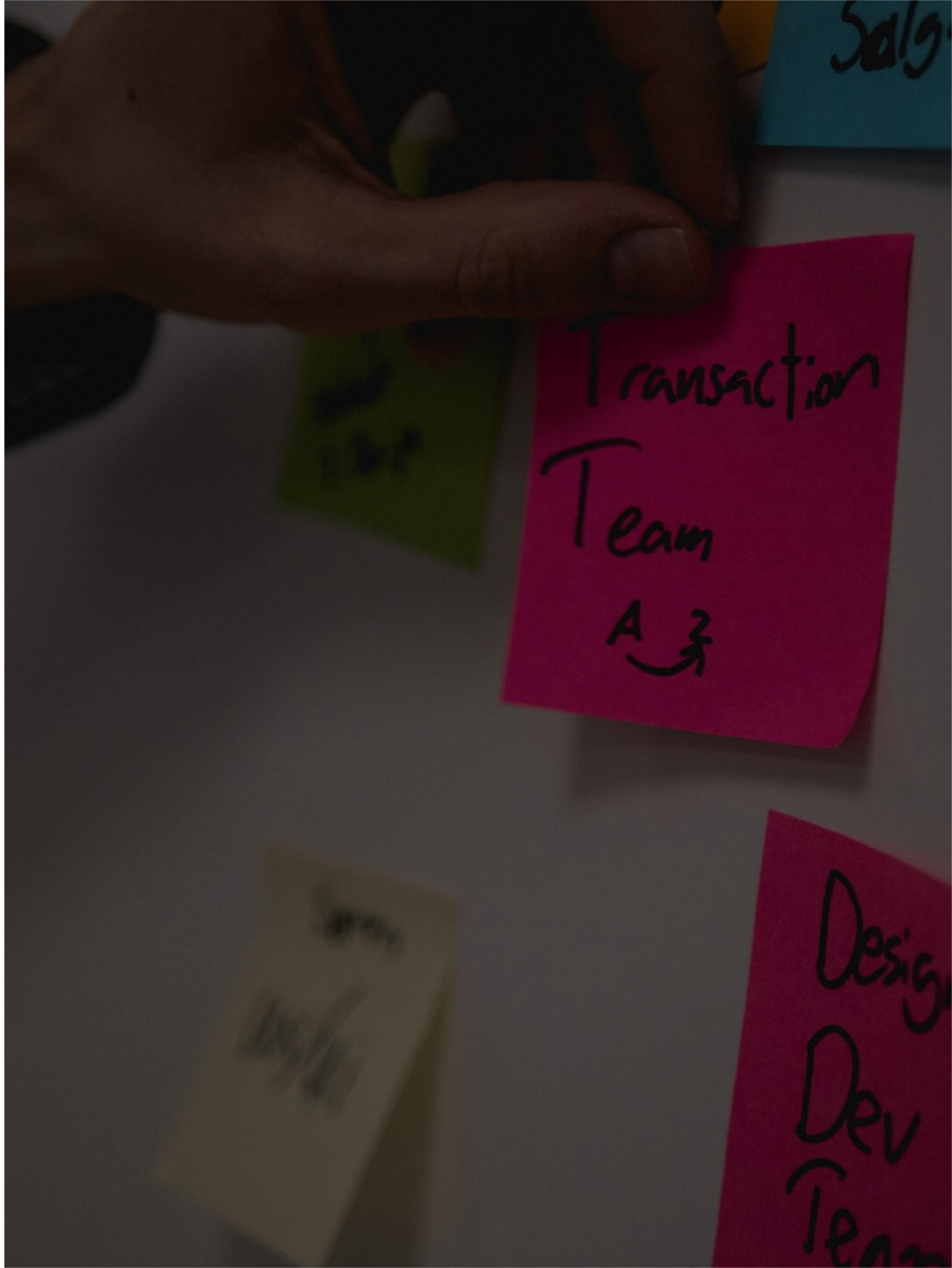
1. **Prompt chains are prototypes, not production systems.** Enterprise needs planning.
2. **GOAP gives you deterministic, explainable orchestration.** The planner finds the path.
3. **Type safety is not optional.** Your domain model IS your contract.
4. **LLMs are tools, not orchestrators.** Let the planner drive.
5. **No new stack required.** Embabel runs on your existing Spring Boot app.

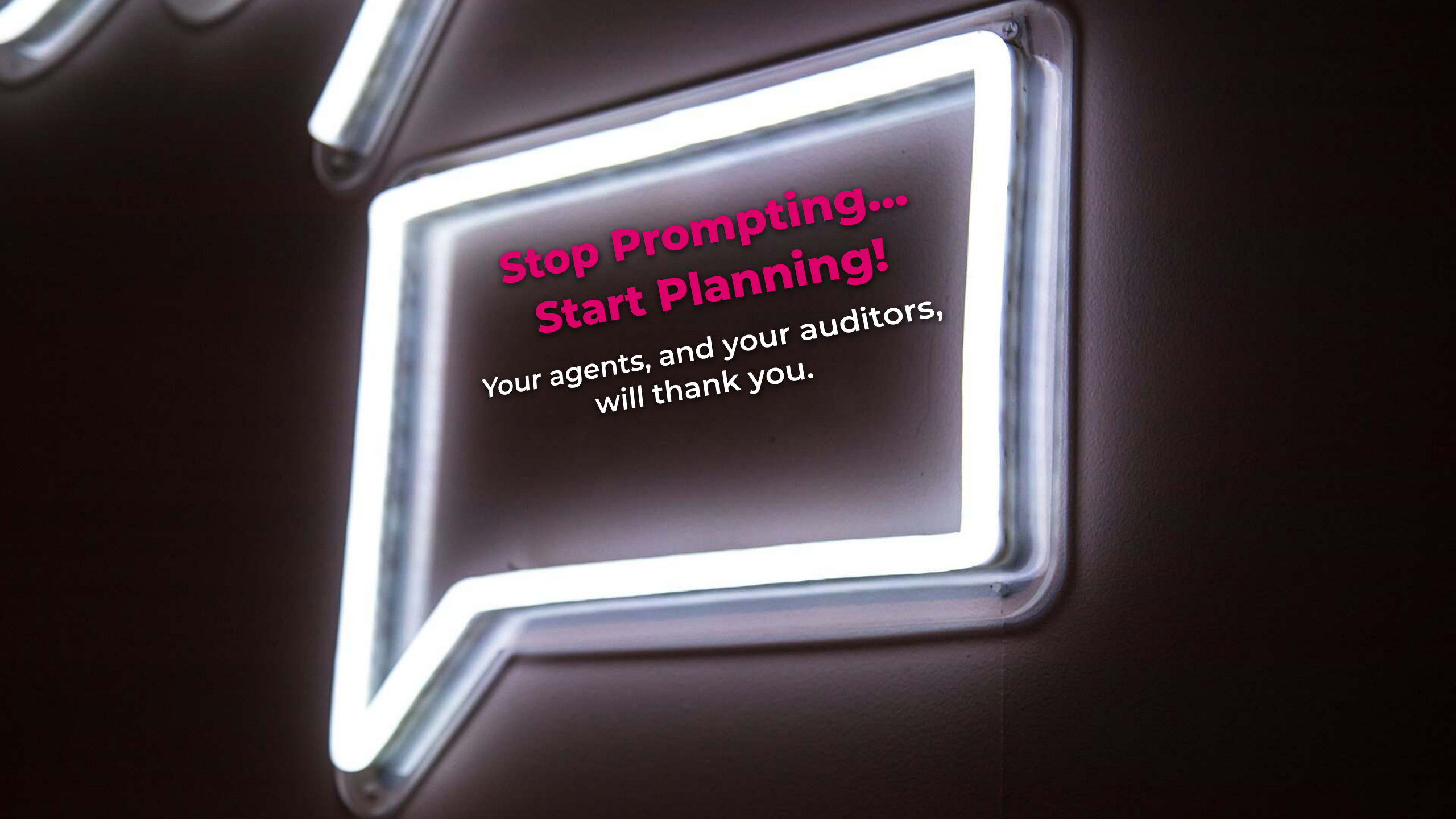
Feedback Loop

Help me make this talk better

- 💡 What was the most useful thing you learned?
- 🤔 What was confusing or could be clearer?
- 🚀 What topic should I go deeper on next time?

No feedback = it works on your machine




Stop Prompting...
Start Planning!
Your agents, and your auditors,
will thank you.


Thank You!
Let's Keep Building!

Stop Prompting, Start Planning

Reliable Agentic AI with Embabel

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

Further Resources

Get Started

- Docs: https://docs.embabel.com
- GitHub: https://github.com/embabel/embabel-agent
- Curated List: https://github.com/embabel/awesome-embabel

Watch

- Rod Johnson - *GenAI Grows Up: Production-Ready Agents on the JVM* (GOTO 2025)
- Dan Vega - *Building AI Agents in Java with Embabel*

Connect

- Embabel Discord

References (1)

Statistics & Industry Reports

- MIT Sloan Management Review - *6 ways businesses can leverage generative AI* (2025)
- MIT NANDA - *State of AI in Business 2025 Report* (2025)
- Gartner - *Top Strategic Technology Trends 2026* (Oct 2025)
- ThoughtWorks - *Technology Radar Vol. 34* (April 2026)
- Anthropic - *Model Context Protocol Tool Use Evaluation* (2025)

Regulatory

- EU AI Act - Regulation (EU) 2024/1689, fully applicable Aug 2025
- DORA - Digital Operational Resilience Act (EU) 2022/2554, applicable Jan 2025
- Air Canada ruling - *Moffatt v. Air Canada*, BCCRT 2024

References (2)

Embabel & GOAP

- Rod Johnson - *Embabel: A New Agent Platform For the JVM* (Medium, 2025)
- Rod Johnson - *GenAI Grows Up: Production-Ready Agents on the JVM* (GOTO 2025)
- Jeff Orkin - *Three States and a Plan: The A.I. of F.E.A.R.* (GDC 2006)

Tools & Frameworks

- Microsoft Presidio: Open-source PII detection
- Model Context Protocol: Anthropic (2024)
- Spring AI